Livraria API — o papel de cada parte (AV2)

O fluxo, em resumo

Quando a API sobe: o Docker compila com o JDK e roda o `.jar` só com o JRE → o **Flyway** aplica as migrations novas (V1 a V5) → o **Hibernate** (`validate`) confere se as entidades batem com as tabelas → o **springdoc** gera a página do Swagger.

Login: `POST /api/auth/register` cria o usuário com a senha em BCrypt (201; e-mail repetido → 409). `POST /api/auth/login` confere a senha e devolve o token JWT (200; errado → 401).

Uma requisição com token (ex: cadastrar livro):

```
POST /api/v1/livros   Authorization: Bearer eyJ...
  JwtFilter ........ lê o token, confere assinatura e expiração, marca o usuário como logado
  SecurityConfig ... rota exige login? se não estiver logado → 401
  LivroController .. @Valid confere o DTO → se inválido, 400
  LivroMapper ...... DTO → entidade (campos simples)
  LivroService ..... busca autor, categoria e editora pelos ids → se não existir, 404; salva
  LivroRepository .. save → o Hibernate gera o INSERT
  LivroMapper ...... entidade → LivroResponseDto
  → 201 Created + JSON do livro
```

Erros: o service (ou o Spring) lança uma exceção e o `GlobalExceptionHandler` escolhe o status.

Família	Significa	No projeto
2xx	deu certo	200 OK, 201 Created, 204 No Content
4xx	erro de quem pediu	400 dado inválido, 401 sem login/token, 404 não existe, 409 já existe ou tem vínculo
5xx	erro do servidor	500 só em bug inesperado

Controller (`controller/`)

- **O que é:** a porta de entrada HTTP. Recebe a requisição e devolve a resposta com o status.
- **Anotações:** `@RestController` (responde JSON), `@RequestMapping("/api/v1/livros")` (prefixo), `@PostMapping` / `@GetMapping` / `@PutMapping` / `@DeleteMapping` (o verbo), `@PathVariable` (o `{id}` da URL), `@RequestBody` (o JSON) e `@Valid` (manda validar o DTO).
- **No projeto:** 5 controllers: Livro, Autor, Categoria, Editora e Auth. Cada CRUD tem 5 rotas: criar, listar, buscar por id, atualizar e excluir.
- **Respostas:** `ResponseEntity.created(...)` → 201 com o cabeçalho `Location`; `ok(...)` → 200; `noContent()` → 204.
- **Frase:** "O controller só cuida do HTTP: recebe, chama o service e devolve o status certo. Não tem regra de negócio."

DTO (`dto/request` e `dto/response`)

- **O que é:** o formato do JSON. O request é o que entra; o response é o que sai.

- **Como é feito:** `record`, uma classe imutável só com os campos, que já vem com construtor e getters (`dto.titulo()`).
- **Validação (só no request):** `@NotBlank` (texto não vazio), `@NotNull` (obrigatório), `@Size(max = 255)` (tamanho), `@Positive` (maior que zero), `@Email` (formato de e-mail).
- **No projeto:** o `LivroRequestDto` recebe os **ids** de autor, categoria e editora; o `LivroResponseDto` devolve os **nomes**. O `@Schema` dá o exemplo que aparece no Swagger.
- **Frase:** "O DTO separa o que o cliente vê da entidade do banco. Controlo o que entra, valido, e nunca exponho dado interno, como a senha."

Mapper (`mapper/`)

- **O que é:** o conversor entre DTO e entidade.
- **Métodos:** `toEntity(dto)` (JSON → entidade), `toResponse(entidade)` (entidade → JSON) e `toResponseList(lista)`.
- **Anotação:** `@Component`, pro Spring criar e injetar.
- **No projeto:** o `LivroMapper.toEntity` copia só título, isbn, ano e preço. Transformar o id do autor em objeto exige ir ao banco, e isso é papel do service. O `toResponse` faz `livro.getAutor().getNome()`.
- **Frase:** "O mapper tira a conversão do controller e do service: cada classe faz uma coisa só."

Service (`service/`)

- **O que é:** a regra de negócio, o "cérebro".
- **Anotação:** `@Service`. As dependências chegam pelo **construtor** (injeção de dependência).
- **No projeto:**
  - `LivroService.criarLivro`: busca autor, categoria e editora pelos ids; se algum não existir, lança a exceção (404).
  - `buscarLivro`: `findById(...).orElseThrow(...)`, que nunca devolve `null`.
  - `excluirLivro`: busca antes de apagar, por isso dá 404 em vez de 204 quando o id não existe.
  - `AuthService`: embaralha a senha com BCrypt no cadastro, confere no login e gera o token.
- **Frase:** "O service tem as regras. Ele não sabe nada de HTTP: quando algo dá errado, só lança uma exceção."

Repository (`repository/`)

- **O que é:** o acesso ao banco.
- **Como é feito:** uma **interface** que estende `JpaRepository<Livro, UUID>` (a entidade e o tipo do id). Você não escreve código: o **Spring Data** cria a implementação.
- **Já vem pronto:** `save`, `findById`, `findAll`, `delete`, `deleteById`, `existsById`...
- **Métodos pelo nome:** no `UsuarioRepository`, o `findByEmail` e o `existsByEmail`. O Spring lê o nome e monta o `SELECT ... WHERE email = ?`.
- **Frase:** "O repository é só uma interface; o Spring Data gera o código. E dá pra criar consulta só pelo nome do método, como `findByEmail`."

Entity / model (`model/`)

- **O que é:** a classe que representa uma tabela. É o **ORM**, feito pelo **Hibernate**, que implementa o JPA.
- **Anotações:** `@Entity` (é uma tabela), `@Id` (chave primária), `@GeneratedValue(strategy = GenerationType.UUID)` (o Java gera o id), `@ManyToOne` com `@JoinColumn(name = "autor_id")` (a ligação com outra tabela).
- **No projeto:** `Livro`, `Autor`, `Categoria`, `Editora` e `Usuario`. O `Livro` tem três `@ManyToOne`.
- **Frase:** "A entidade mapeia a tabela; o Hibernate gera o SQL. No `Livro`, o `@ManyToOne` liga ao autor: no Java eu uso o objeto, no banco fica o id."

GlobalExceptionHandler (`exception/`)

- **O que é:** o lugar central que transforma exceção em resposta HTTP.
- **Anotações:** `@RestControllerAdvice` (vigia todos os controllers) e `@ExceptionHandler(XException.class)` (um método por tipo de erro).
- **Tabela:** `MethodArgumentNotValidException` → 400 · `...NaoEncontradoException` → 404 · `CredenciaisInvalidasException` → 401 · `EmailJaCadastradoException` e `DataIntegrityViolationException` → 409 · qualquer outra → 500.
- **Formato:** todo erro sai igual: `timestamp`, `status`, `error`, `message` e `path` (o `ErrorResponseDto`).
- **Frase:** "Concentro o tratamento de erro num lugar só, e toda resposta de erro tem o mesmo formato."

JwtService (`security/`)

- **O que é:** a fábrica e o conferidor de tokens.
- **`gerarToken(email)`:** coloca o e-mail no `sub`, a data de criação (`iat`) e a de expiração (`exp`, 1 hora, vinda do yaml), e **assina** com a secret usando HS256.
- **`extrairEmail(token)`:** recalcula a assinatura e confere a expiração. Se o token foi alterado ou venceu, **lança exceção**; se está tudo certo, devolve o e-mail.
- **A secret:** vem do `application.yaml` (`jwt.secret`), lida de uma variável de ambiente.
- **Frase:** "O JwtService gera o token no login e confere a assinatura e a validade em cada requisição."

JwtFilter (`security/`)

- **O que é:** o porteiro que roda **antes de toda requisição**.
- **Como é feito:** estende `OncePerRequestFilter`, então roda uma vez por requisição.
- **Passos:** pega o cabeçalho `Authorization`, tira o `"Bearer "`, valida pelo `JwtService` e, se for válido, coloca o usuário no `SecurityContextHolder`. Depois **sempre** deixa seguir.
- **Importante:** ele **não barra** ninguém; só marca quem está logado.
- **Frase:** "O filtro lê o token e marca o usuário como autenticado. Quem decide se pode passar é o SecurityConfig."

SecurityConfig (`config/`)

- **O que é:** as regras de segurança.
- **O que define:**

    - `csrf.disable()`: não se aplica, porque a API não usa cookie de sessão;
    - `STATELESS`: sem sessão no servidor;
    - `/api/auth/**` e Swagger → `permitAll()`; todo o resto → `authenticated()`;
    - quem não está logado → **401**;
    - o `JwtFilter` entra **antes** do filtro padrão;
    - o `PasswordEncoder` é o BCrypt.
- **Frase:** "Aqui ficam as regras: login e Swagger abertos, o resto exige token, sem sessão no servidor."

Flyway (`resources/db/migration/`)

- **O que é:** o versionamento do banco.
- **Como funciona:** os arquivos `V1__...sql` a `V5__...sql` rodam em ordem, uma vez só, e ficam anotados na `flyway_schema_history`. Migration aplicada não se edita: mudou, cria a próxima.
- **Ligação com o Hibernate:** `ddl-auto: validate`. O Hibernate só confere; quem altera o banco é o Flyway.
- **Frase:** "Cada mudança no banco é um SQL versionado no Git, que roda igual em qualquer máquina."

Swagger (`config/OpenApiConfig` + anotações)

- **O que é:** a documentação interativa, gerada **a partir do código** pelo **springdoc**.
- **Onde:** `localhost:8080/swagger-ui/index.html`. O JSON da especificação fica em `/v3/api-docs`.
- **Anotações:** `@Tag` (agrupa), `@Operation` (título da rota), `@ApiResponses` (status possíveis), `@Schema` (descrição e exemplo dos campos).
- **`OpenApiConfig`:** título da API e o botão **Authorize**, onde se cola o token.
- **Frase:** "A documentação nasce do código, então nunca desatualiza. Uma rota nova aparece sozinha."

Docker (`Dockerfile`, `.dockerignore`, `compose.yaml`)

- **O que é:** empacota a aplicação com tudo que ela precisa pra rodar igual em qualquer máquina.
- **Dockerfile em duas etapas:** a 1ª compila com o **JDK** e gera o `.jar`; a 2ª roda só com o **JRE** alpine, que é menor. As dependências são baixadas antes do código, por causa do cache, e o container roda com **usuário não-root**.
- **`.dockerignore`:** o que não entra na imagem (`target`, `.git`...).
- **`compose.yaml`:** guarda a configuração (montar a partir desta pasta, porta 8080). Um comando sobe tudo: `docker compose up --build`.
- **Imagem x container:** a imagem é o molde, o container é ela rodando, como classe e objeto.
- **Frase:** "Com Docker, a API roda igual em qualquer máquina, sem instalar Java. A imagem final é pequena porque só leva o JRE e o `.jar`."